

Serverless Architecture on AWS

Hands-On Lab Guide — Sections 4 & 5

A practical, step-by-step laboratory walkthrough for building event-driven microservice backends using Amazon API Gateway, AWS Lambda, AWS Elastic Beanstalk, and Amazon EventBridge — all without provisioning or managing a single virtual server.

AWS ACADEMY · ITNW1009

75 MIN PER LAB



What You Will Build Across Both Labs



Section 4: API Gateway + Lambda

Author a live Node.js serverless function and expose it through a public REST API endpoint. Validate end-to-end with a real HTTP POST request.



Section 5 Part A: Elastic Beanstalk

Explore a pre-built managed PaaS environment, observe auto-scaling configuration, and deploy a source code update without touching any underlying infrastructure.



Section 5 Part B: Lambda + EventBridge

Write a Python Boto3 automation script and wire it to a scheduled EventBridge cron rule that automatically stops a running EC2 instance every 60 seconds.

Section 4 — Lab Overview

In this lab you will build a complete serverless API backend from scratch. An **Amazon API Gateway** REST endpoint acts as the public-facing front door, routing incoming HTTP client traffic directly to a custom **Node.js** function running inside **AWS Lambda** — no EC2 instances, no web servers, no infrastructure to maintain.

Author

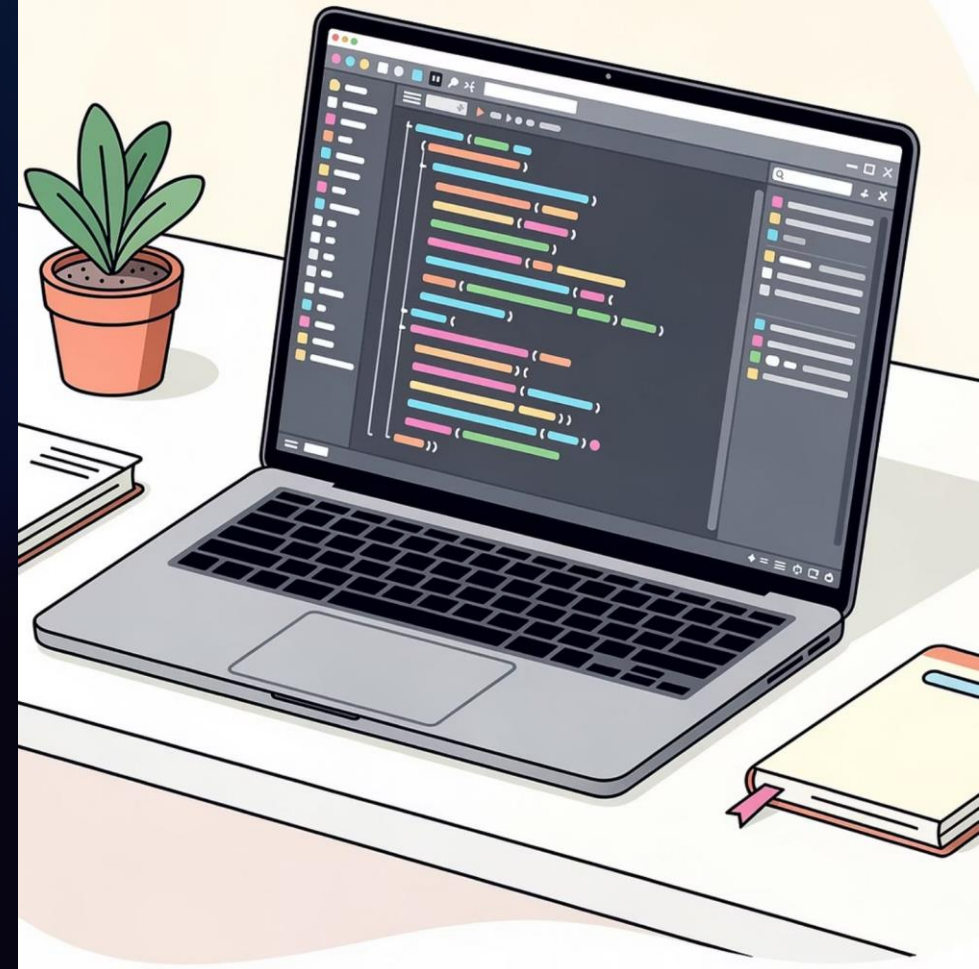
A Lambda function with custom Node.js application logic

Configure

An API Gateway REST endpoint mapped to your function

Verify

End-to-end connectivity with a live HTTP POST test



Section 4 — Learning Objectives

1 Navigate the AWS Console

Create and configure an isolated AWS Lambda function using the Management Console dashboard.

2 Map Execution Permissions

Assign the pre-provisioned **LabRole** IAM security profile to grant your function the necessary runtime permissions.

3 Build a REST API Entrypoint

Construct an Amazon API Gateway REST API POST method integrated with your serverless backend.

4 Validate Integration

Execute end-to-end connectivity tests and analyze real-time execution metrics inside CloudWatch.

Prerequisites & Safety Rules

- ⊗ Never create custom IAM Users, Groups, Roles, or modify root billing settings. Doing so triggers an AWS Academy SCP restriction and may lock your learner account.

Platform Environment

AWS Academy Curriculum Provided Guided Lab — Course Pre-Packaged Sandbox or Module Lab Environment.

- Duration: **75 Minutes**
- All services run inside an isolated sandbox container
- Do not attempt to access billing or root configuration layers

The LabRole Rule

Every service asset you create in this lab **must** use the pre-provisioned IAM security profile named **LabRole**. This role has all necessary administrative permissions pre-verified for the target services.

- Do not create a new role from scratch
- Do not select any other role from the dropdown
- LabRole appears in every relevant configuration panel

Architecture Overview — Section 4



A public HTTP client sends a POST request to your API Gateway endpoint. The gateway translates the request and triggers your Lambda function, which runs the Node.js handler under LabRole permissions and returns a structured JSON response — all without a single managed server.

STEP 1 OF 5

Initialize the AWS Infrastructure Container

01

Log In to AWS Academy

Open your browser and sign in to the **AWS Academy Learning Management Portal**. Navigate to your current course enrollment.

03

Wait for Green

The status circle will blink **amber** during initialization. Wait until it turns a steady **green** — this confirms your isolated AWS environment is fully ready.

02

Launch the Sandbox

Select **Day 6: Compute Services Infrastructure Sandbox** and click **Start Lab** in the top control header. Watch the status indicator.

04

Open the Console

Click the **AWS** link next to the green indicator. A new browser tab will launch, auto-authenticating your session into the live **AWS Management Console**.

STEP 2 OF 5

Create the Lambda Function

In the AWS Console search bar, type Lambda and open the service. Click the orange **Create function** button. Select **Author from scratch** and enter these exact settings:

Field	Value
Function name	BriefTextConverterService
Runtime	Node.js 18.x (or latest active LTS)
Architecture	x86_64 (default)
Execution role	Use an existing role → LabRole

Expand **Change default execution role**, select **Use an existing role**, and choose **LabRole** from the dropdown. Click the orange **Create function** button and wait 10–15 seconds for provisioning to complete.

STEP 3 OF 5

Author the Node.js Application Code

Once your function dashboard loads, scroll to the **Code source** panel. Double-click `index.mjs` to open it. **Delete all existing boilerplate**, then paste the script below:

```
export const handler = async (event) => {
  console.log("Event Received:", JSON.stringify(event, null, 2));

  let clientUserName = "Guest Professional Analyst";
  if (event.body) {
    try {
      const parsedBody = JSON.parse(event.body);
      if (parsedBody.name) { clientUserName = parsedBody.name; }
    } catch (error) {
      console.log("Non-JSON payload. Using defaults.");
    }
  }

  const operationalResponsePayload = {
    status: "SUCCESS_VALIDATION_COMPLETE",
    message: `Hello, ${clientUserName}! Your event executed in AWS Lambda.`,
    timestamp: new Date().toISOString(),
    environment: "AWS_Academy_ITNW1009_Sandbox"
  };

  return {
    statusCode: 200,
    headers: {
      "Content-Type": "application/json",
      "Access-Control-Allow-Origin": "*"
    },
    body: JSON.stringify(operationalResponsePayload)
  };
};
```



After pasting, click the orange **Deploy** button above the code editor. Wait for the confirmation overlay — your function is now live on the cloud.

What the Node.js Code Does

1 — Logs the Incoming Event

`console.log` writes the full raw event payload to **Amazon CloudWatch Logs** for real-time debugging and audit tracing.

2 — Parses the Request Body

The handler attempts to parse a JSON body. If a `name` key is present, it replaces the default value *"Guest Professional Analyst"* with the caller's supplied name.

3 — Builds the Response Object

A structured JSON payload is assembled with a status flag, a personalized message, a server-side timestamp, and the sandbox environment label.

4 — Returns an HTTP 200 Response

The function returns a properly formatted HTTP response with `statusCode: 200`, JSON content-type headers, and CORS support via `Access-Control-Allow-Origin: *`.

Create the API Gateway REST API

In the AWS Console search bar, type `API Gateway` and open the service. Scroll to the service selection matrix and click **Build** next to **REST API**. Do **not** select the Private or HTTP variants.

Field	Value
Create	New API
API name	<code>LegalServicePublicIngressAPI</code>
Description	Public gateway endpoint routing traffic serverlessly to Lambda
Endpoint Type	Regional(default)

Click the blue **Create API** button. The console will load your new API's Resources tree view, ready for method configuration.

Configure the POST Method Integration

1

Create Method

In the Resources tree, click **Actions** → **Create Method**. Select **POST** from the dropdown and click the checkmark.

2

Set Integration Type

Choose **Lambda Function** as the integration type. Set your Lambda Region (e.g., `us-east-1`).

3

Enable Proxy Integration

Check **Use Lambda Proxy integration**. This passes the full raw HTTP request into your Node.js event object — critical for body parsing.

4

Link the Function

Type `BriefTextConverterService` in the Lambda Function field, confirm the autocomplete selection, and click **Create Method**.

 Lambda Proxy Integration is mandatory. Without it, the raw HTTP body will not reach your Node.js handler and the API will return a 502 error.

STEP 5 OF 5

Deploy the API to a Live Production Stage

Deployment Settings

Deployment stage	[New Stage]
Stage name	prod
Stage description	Active vocational learner lab production stage

After You Click Deploy

The console loads your **Stage Editor** dashboard. At the very top of the blue panel, locate the field labeled **Invoke URL**. It will look like:

```
https://a1b2c3d4e5.execute-api.us-east-1.amazonaws.com/prod
```

Copy this entire URL to your clipboard — you will use it in the validation task next.

Task A — Live API Payload Test

Open a second browser tab and navigate to a free online HTTP testing tool (such as an online curl executor or webhook playground), or use your local terminal if you have command-line experience. Configure a **POST** request as follows:

Target URL

Paste your copied **Invoke URL** into the destination address field.

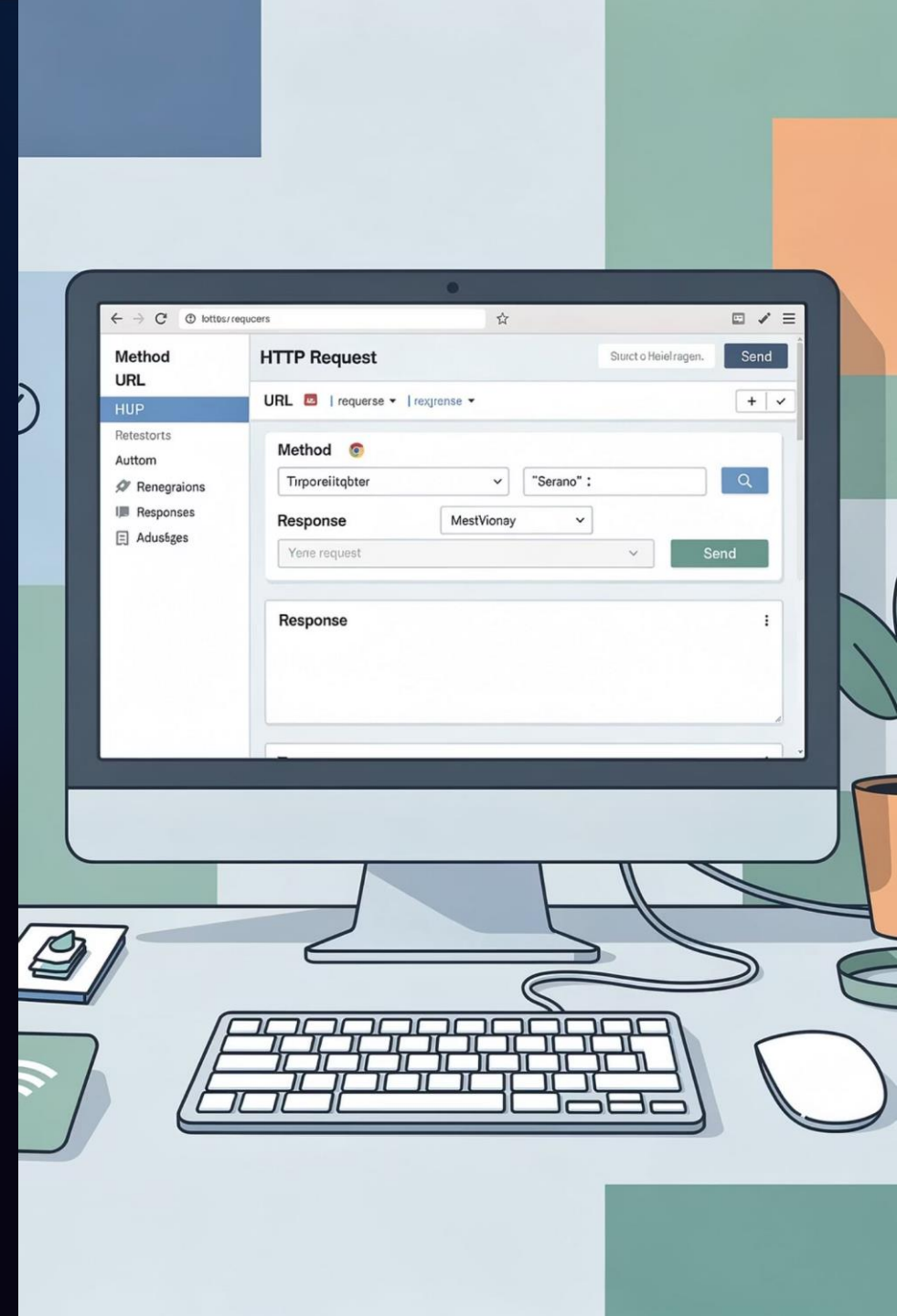
Content Type

Set the body data type to `application/json`.

Request Body

```
{"name":  
  "Professor  
  Vance"}
```

Expected Result: The response status must be **200 OK**. The JSON body must contain `"status": "SUCCESS_VALIDATION_COMPLETE"` and the message: *"Hello, Professor Vance! Your legal document text-parsing event string executed successfully inside AWS Lambda without an active server host."*



Task B — Lab Environment Checkpoint Verification

01

Return to AWS Academy

Navigate back to your primary **AWS Academy Course Instructions** tab in the browser.

03

Run the Checkpoint Script

Click **Run Active Checkpoint Environment Evaluation Script** and wait for the automated checker to complete its scan.

02

Open the Progress Verifier

Locate the **Lab Environment Progress Verifier** sidebar panel on the right side of the instruction page.

04

Confirm Green Checkmarks

Verify that the tracking panel logs show **successful checkmark indicators** for your Lambda function, IAM role assignment, and API Gateway POST method configuration.

Section 4 — Troubleshooting Guide

Observed Error	Root Cause	Remediation
Access Denied banner during function creation	Attempted to create a custom IAM role instead of using LabRole	Reopen Lambda config → expand <i>Change default execution role</i> → select Use an existing role → choose LabRole
API returns 502 Bad Gateway	Lambda Proxy Integration checkbox was not enabled on the POST method	API Gateway → Resources → POST → Integration Request → enable Use Lambda Proxy integration → save → redeploy to prod
API returns 504 Timeout or hangs	Code syntax error or function was not Deployed after editing	Lambda Code editor → verify <code>index.mjs</code> matches guide exactly → click Deploy above the editor

Section 4 Complete ✓

✓ Lambda Function

`BriefTextConverterService` authored,
deployed, and running under `LabRole`

✓ REST API

`LegalServicePublicIngressAPI`
POST method with Proxy Integration
deployed to `prod` stage

✓ Validated

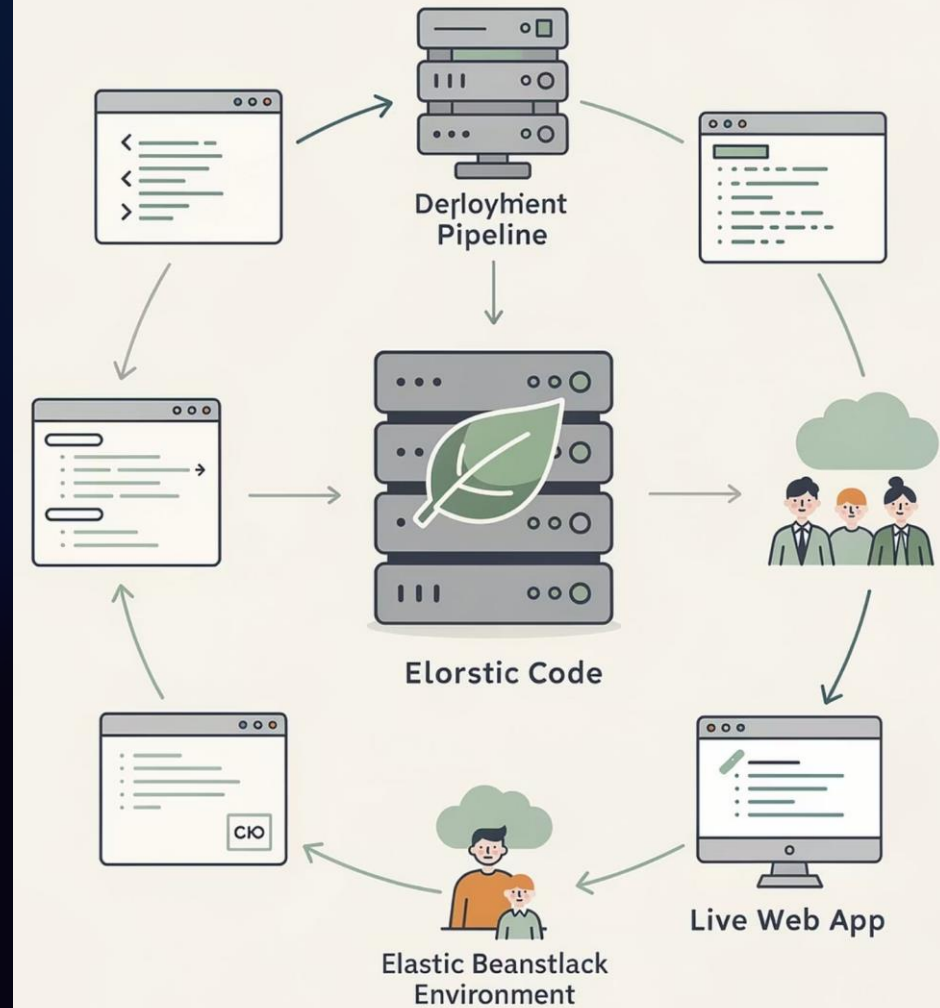
Live HTTP POST returned 200 OK with
correct JSON payload and checkpoint
script confirmed green

You have successfully built and validated a fully serverless API backend on AWS — no EC2, no web server, no infrastructure management required. Proceed to **Section 5**.

Section 5 — Lab Overview

Lambda, Elastic Beanstalk & EventBridge

This lab has three distinct parts. You will explore managed PaaS hosting with **Elastic Beanstalk**, write a Python infrastructure automation script with **Lambda**, and orchestrate a live scheduled cron trigger with **Amazon EventBridge** — combining all three services into a complete automation pipeline.



Section 5 — Learning Objectives

1 Deploy with Elastic Beanstalk

Launch and inspect a fully managed PaaS web application environment with auto-scaling, load balancing, and health monitoring built in.

2 Deploy Code Updates

Upload and deploy application source code updates to a live environment without configuring web servers, load balancers, or scaling groups.

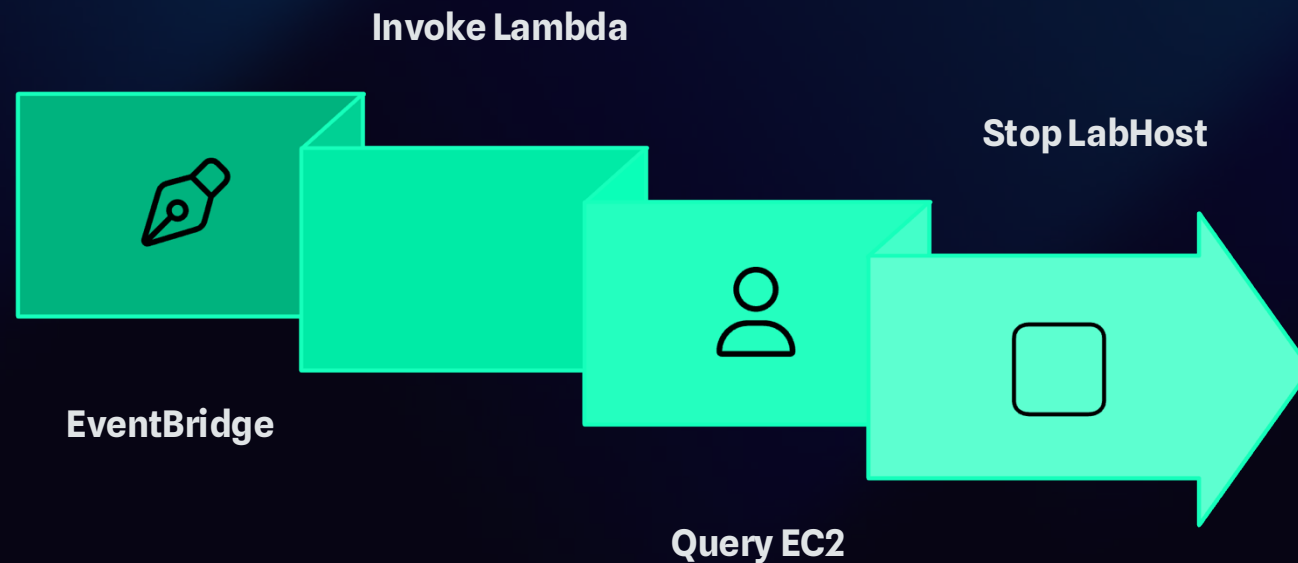
3 Author a Python Lambda Function

Write an AWS Boto3 Python script that programmatically reads EC2 instance state and issues automated stop commands.

4 Schedule with EventBridge

Build a rate-based EventBridge cron rule that triggers your Lambda function every 60 seconds to enforce automated compute lifecycle management.

Section 5 — Lab Architecture

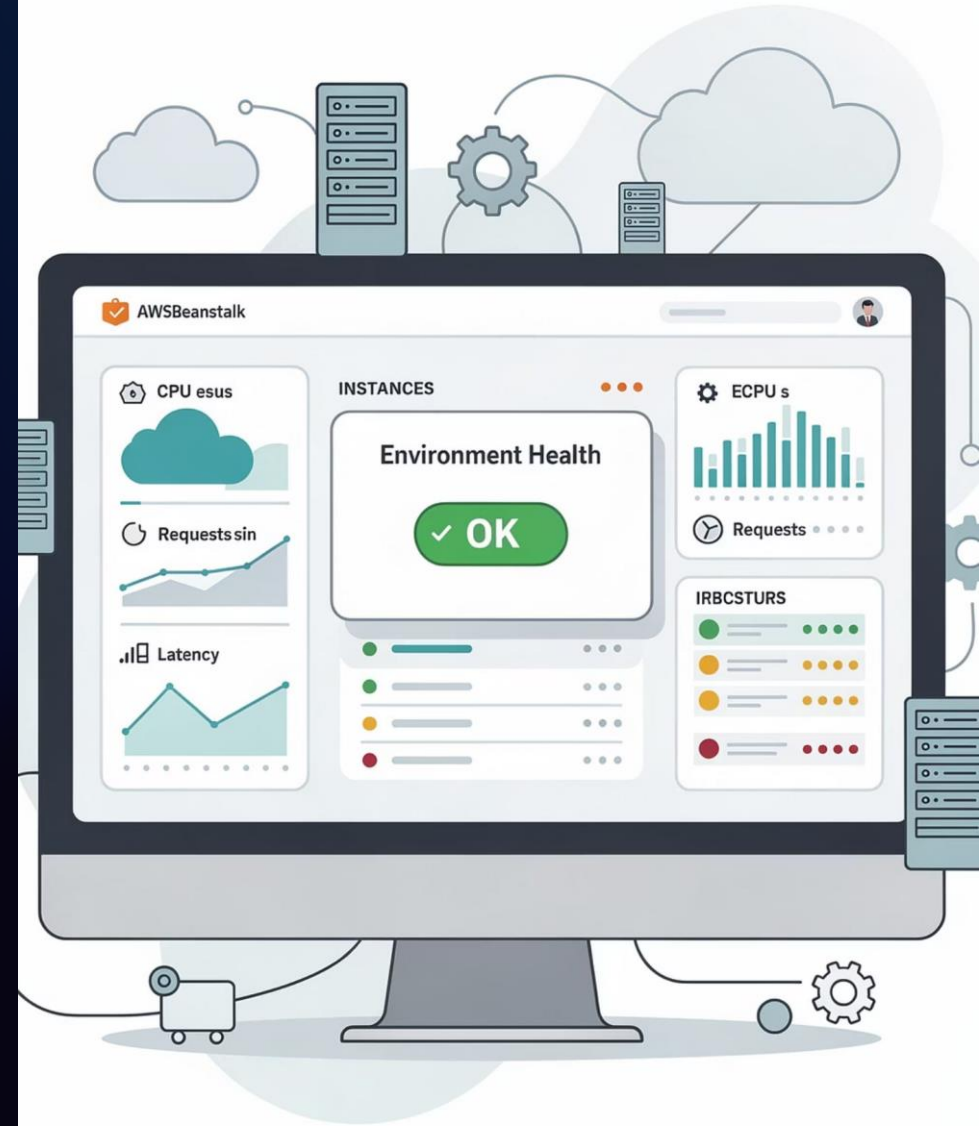


An EventBridge scheduled rule fires on a 1-minute cadence, invoking the Python Lambda function. The function uses the Boto3 SDK to query the state of a target EC2 instance and automatically issues a stop command if it is found running — eliminating idle compute costs without any manual intervention.

SECTION 5 · PART A

Managed PaaS via AWS Elastic Beanstalk

Elastic Beanstalk is AWS's **Platform-as-a-Service** offering. You provide the application code; AWS handles the OS, runtime, web server, load balancer, auto-scaling, and health monitoring automatically. In Part A, you will explore a pre-built environment and deploy a source code update — all without a single CLI command.



Initialize & Explore the Beanstalk Environment

01

Open Elastic Beanstalk

In the AWS Console search bar, type `Elastic Beanstalk` and select the service. In the left navigation panel, click on the environment labeled **Sampleapp-env** (or the name specified in your module instructions).

03

Browse Configuration

Click **Configuration** in the left sidebar. Review how the platform automatically specifies EC2 security groups, instance types (`t3.micro`), and min/max scaling boundaries (e.g., 2–6 instances) — no CLI scripts required.

02

Verify Health Status

Confirm the health ring icon shows a **steady green — OK** status. Any other color indicates the environment is still initializing or has a configuration issue to resolve first.

04

View Monitoring Charts

Click **Monitoring** in the left panel. Observe auto-generated charts tracking CPU utilization, request latency, and HTTP error counts — all built natively into the platform dashboard.

What Elastic Beanstalk Manages For You

**1. EC2
Instance
Provisioning**

**2. OS and
Runtime
Patching**

**3. Load
Balancer
Configuration**

**4.
Auto-Scaling
Rules**

**5. Health
Monitoring
and Alerts**

**6. Rolling
Deployment
Coordination**

When you click **Configuration** → **Data tier**, notice that adding a full database layer requires only selecting a few option boxes – the managed service handles all underlying storage provisioning automatically. Click **Cancel** to exit without saving changes.

STEP A2

Deploy an Application Source Code Update

Upload & Deploy

From the environment overview dashboard, click **Upload and deploy**. In the prompt window:

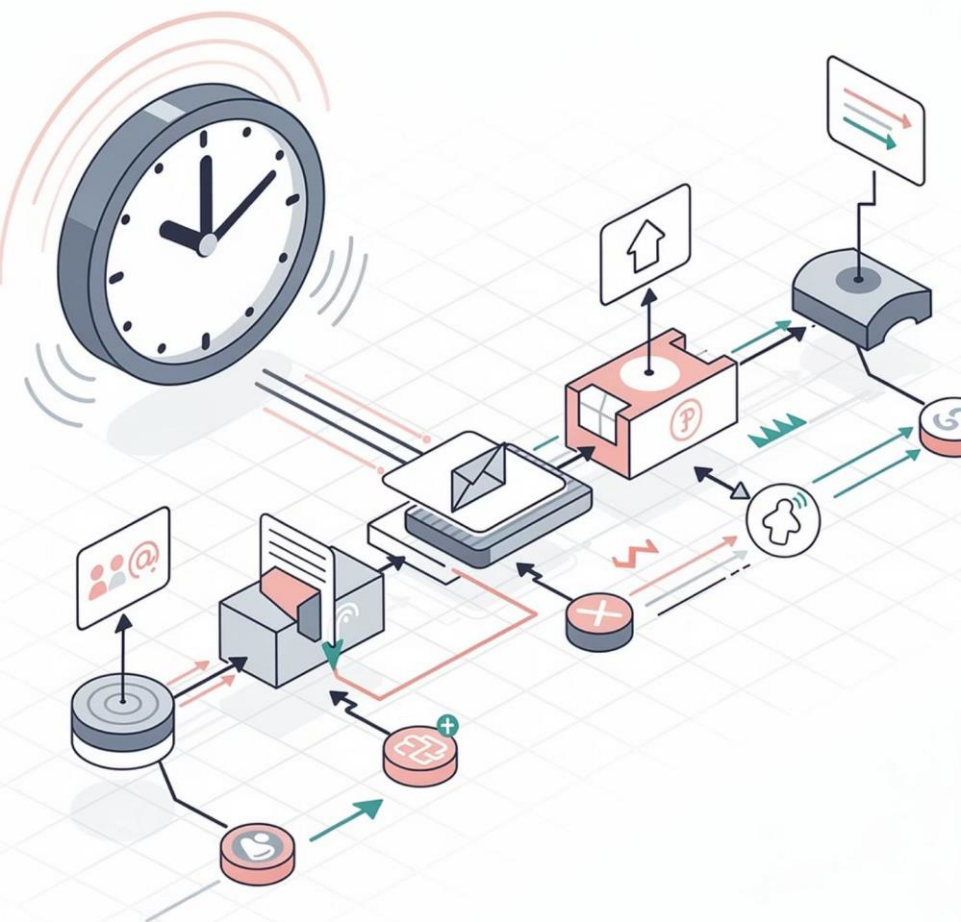
- Click **Choose file** and select the `sample-v2-update.zip` archive provided by your instructor
- In the Version label field, enter: `v2-vocational-update-release`
- Click the blue **Deploy** button

What to Expect

The health circle will temporarily turn **amber/grey** with the message *"Updating environment..."* while the platform coordinates rolling file updates across active hosts, refreshes routing maps, and restarts web server processes.

Wait approximately **2 minutes** until the health ring returns to a steady **green – OK**. Then click the URL at the top of the dashboard to verify the updated app loads in the browser.

Serverless Automation via Lambda & EventBridge



STEP B1

Locate & Record the Target EC2 Instance

01

Open the EC2 Console

In the AWS Console search bar, type **EC2** and select the service. In the left navigation pane, click **Instances**.

03

Copy the Instance ID

Click the checkbox next to **LabHost**. In the lower details panel, locate the **Instance ID** field (e.g., `i-0a1b2c3d4e5f6g7h8`). Copy this exact alphanumeric token to your clipboard.

02

Locate LabHost

Find the pre-provisioned compute instance labeled **LabHost**. Confirm its state reads a solid **green – Running** before proceeding.

04

Save the ID

Paste the Instance ID into a temporary local text document. You will need this exact string when coding your Lambda function in the next step – do not lose it.

STEP B2

Create the Python Lambda Function

Navigate back to the **Lambda** service. Click **Create function**. Select **Author from scratch** and enter the following exact configuration values:

Field	Value
Function name	AutomatedInstanceStopController
Runtime	Python 3.9 (or latest active LTS)
Execution role	Use an existing role → LabRole

Expand **Change default execution role**, select **Use an existing role**, and choose **LabRole**. Scroll to the bottom and click the orange **Create function** button. Wait for the dashboard to confirm successful provisioning.

STEP B3

Code the Python Boto3 Automation Script

Scroll to the **Code source** panel. Open `lambda_function.py` in the editor. Delete all boilerplate code and paste the script below. Then **replace the placeholder Instance ID** with your real value from Step B1:

```
import boto3
import json

ec2_client_connection = boto3.client('ec2', region_name='us-east-1')

# CRITICAL: Replace with your real Instance ID from Step B1
TARGET_COMPUTE_INSTANCE_ID = 'i-0a1b2c3d4e5f6g7h8'

def lambda_handler(event, context):
    print(f"Triggered. Checking state for: {TARGET_COMPUTE_INSTANCE_ID}")
    try:
        query = ec2_client_connection.describe_instances(
            InstanceIds=[TARGET_COMPUTE_INSTANCE_ID])
        state = query['Reservations'][0]['Instances'][0]['State']['Name']
        print(f"Current state: {state}")
        if state == 'running':
            ec2_client_connection.stop_instances(
                InstanceIds=[TARGET_COMPUTE_INSTANCE_ID])
            return {'statusCode': 200,
                    'body': json.dumps(f"Stopped instance {TARGET_COMPUTE_INSTANCE_ID}")}
        else:
            return {'statusCode': 200,
                    'body': json.dumps(f"No action needed. State: {state}")}
    except Exception as e:
        print(f"Error: {str(e)}")
        raise e
```



Do not delete the single quote characters surrounding the Instance ID string. Replace only the placeholder text `i-0a1b2c3d4e5f6g7h8` with your real ID. Then click **Deploy**.

What the Python Script Does

Initializes the Boto3 EC2 Client

Creates a native AWS SDK connection to the EC2 service in the `us-east-1` region. This client handles all API calls to the EC2 control plane.

Queries the Instance State

Calls `describe_instances()` with the target Instance ID to retrieve current state metadata — running, stopped, stopping, etc.

Issues a Conditional Stop

If the instance is in a `running` state, the script calls `stop_instances()` to initiate an automated shutdown — eliminating idle capacity costs.

Logs Every Action to CloudWatch

Every `print()` statement feeds into CloudWatch Logs, giving you a complete audit trail of every trigger cycle and state decision.

STEP B4

Create the EventBridge Scheduled Cron Rule

In the AWS Console search bar, type `EventBridge` and select the service. Under the **Amazon EventBridge Scheduler** category, click **Create schedule**. Enter these exact configuration values:

Field	Value
Schedule name	<code>OneMinuteComputeHostCheckerCron</code>
Schedule pattern	Recurring schedule
Schedule type	Rate-based schedule
Rate value	1
Rate unit	Minutes

Configure the EventBridge Target & Permissions

01

Click Next — Select Target

On the **Select target** screen, click **AWS Lambda** in the service targets grid to filter the choices.

03

Set Execution Role

In the **Settings** page, expand **Execution role**, select **Use an existing role**, and choose **LabRole** from the dropdown list.

02

Choose Your Function

In the Lambda function dropdown, select **AutomatedInstanceStopController**. Click **Next**.

04

Review & Create

Click **Next** to reach the Review page. Verify all values match the guide, then click the blue **Create schedule** button. Your automation pipeline is now live.

SECTION 5 · PART C

Verification & Automation Auditing

With your EventBridge cron rule active and Lambda function deployed, it is time to observe the automation in action. Part C covers two verification tasks: watching the EC2 instance stop automatically in real time, and auditing the execution logs in CloudWatch to confirm the script ran as designed.



TASK C1

Watch the Instance Stop Automatically

Open EC2 Instances

In a new tab, open the EC2 console and navigate to **Instances**. Locate **LabHost**.

1

Manually Restart It

Select **LabHost**, click **Instance State** → **Start Instance**. Watch it spin back up to **Running**.

2

3

Observe the State Change

Within **2 minutes**, the EventBridge cron triggers Lambda. Watch **LabHost** transition from **Running** → **Stopping** → **Stopped** without any manual action.

4

Automation Re-Triggers

Within **60 seconds**, the cron fires again and forces the instance back to **Stopped** automatically – demonstrating continuous enforcement.

TASK C2

Audit Serverless Logs in CloudWatch

Navigate to Monitor Tab

Open the **AutomatedInstanceStopController** Lambda function.
Click the **Monitor** tab near the top of the function details page.
Review the automated telemetry charts showing:

- Invocation count spikes
- Error count lines
- Execution duration per invocation

View the Log Stream

Click **View CloudWatch Logs**. Select the latest **Log Stream** timestamp record from the table.
Confirm you can see these log entries in sequence:

1. `Triggered. Checking state for: i-0abc...`
2. `Current state: running`
3. `Stopped instance i-0abc...`

These logs confirm your automation pipeline executed correctly end-to-end.

Key Concepts Reinforced — Serverless vs. Managed PaaS



SERVERLESS LAMBDA

- NO SERVER MANAGEMENT. EVENT-DRIVEN.
- PAY-PER-INVOCATION. AUTO-SCALES TO ZERO.



**MANAGED PAAS
BEANSTALK**



- HANDLES OS & RUNTIME. ALWAYS-ON.
- AUTO-SCALING WITH LIMITS. ZIP UPLOAD DEPLOY.



TRADITIONAL EC2

- FULL CONTROL. MANUAL OS PATCHING.
- MANUAL SCALING. PERSISTENT BILLING.

Understanding when to choose Lambda versus Elastic Beanstalk versus raw EC2 is a foundational skill for cloud architects. These labs demonstrate all three models in a single session.

The LabRole — Why It Matters



Pre-Verified Permissions

LabRole contains all permissions needed for Lambda, API Gateway, EC2, Beanstalk, and EventBridge — pre-configured and pre-audited for this curriculum.



SCP Protection

AWS Academy applies Service Control Policies that block attempts to create custom IAM roles. Always use LabRole to avoid triggering an Access Denied lockout.



Consistent Across All Steps

Whether creating a Lambda function, an EventBridge rule, or an EC2 operation, LabRole is the single correct answer for every "Execution role" field in both labs.

Lambda Proxy Integration — Deep Dive

What It Does

When **Use Lambda Proxy Integration** is enabled on your API Gateway POST method, the gateway passes the *complete, unmodified* HTTP request — headers, query parameters, path parameters, and raw body — directly into the Lambda `event` object.

Your Node.js handler can then access `event.body`, `event.headers`, and `event.queryStringParameters` directly without any mapping templates.

Without Proxy Integration

Without this checkbox, API Gateway expects you to write **Velocity Template Language (VTL) mapping templates** to manually transform the request before it reaches Lambda — an advanced configuration step that is out of scope for this lab.

- ⊗ Skipping this checkbox is the #1 cause of 502 errors in this lab. Always verify it is checked before saving the method integration.

Boto3 SDK — The Python Bridge to AWS

Boto3 is the official AWS Software Development Kit (SDK) for Python. It provides Python classes and methods that map directly to AWS API calls — enabling you to manage virtually any AWS service programmatically from inside a Lambda function.

boto3.client()

Creates a low-level service client. Used in this lab for EC2 operations like `describe_instances()` and `stop_instances()`.

No Installation Needed

Boto3 is pre-installed in all AWS Lambda Python runtimes. You can `import boto3` immediately without any package management steps.

Region Awareness

Always specify `region_name='us-east-1'` (or your current region) when initializing the client to ensure API calls are routed to the correct AWS data center.



Amazon EventBridge — Scheduled Automation



Rate-Based Schedules

A **rate-based schedule** fires at a fixed interval, such as every 1 minute, every 5 minutes, or every 24 hours. It is the simplest way to run recurring automation tasks without cron syntax.



Targets

EventBridge can invoke over 200 AWS service targets. In this lab, your schedule targets a **Lambda function** directly, passing it a small JSON event payload on each invocation cycle.



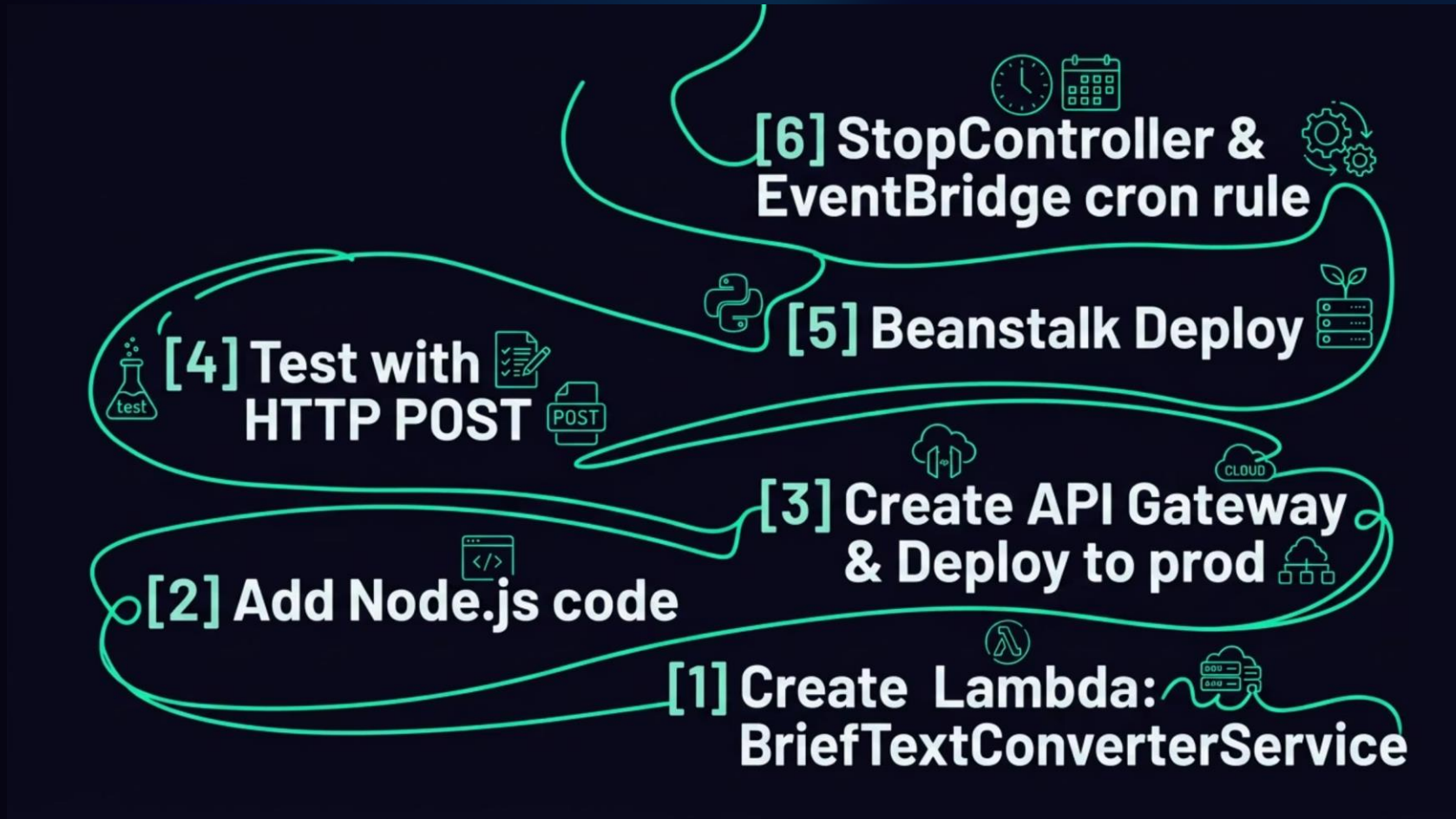
Real-World Use Case

Scheduled Lambda + EventBridge is a common pattern for **automated cost governance** — shutting down dev/test environments outside business hours to eliminate unnecessary EC2 billing.

Troubleshooting — Section 5

Observed Error	Root Cause	Remediation
Lambda execution fails with InvalidInstanceID.NotFound	The placeholder Instance ID was not replaced with the real LabHost Instance ID	Return to <code>lambda_function.py</code> , update <code>TARGET_COMPUTE_INSTANCE_ID</code> with the real ID from Step B1, and click Deploy
EventBridge rule shows no invocations after 5 minutes	Execution role was not set to LabRole, or the schedule was created in a disabled state	Open the schedule in EventBridge, verify the execution role is LabRole , and confirm the schedule status is Enabled
Beanstalk deployment stuck in amber/grey for more than 10 minutes	Upload file was corrupted or version label contained invalid characters	Re-download the zip archive from your instructor, use a simple version label like <code>v2-update</code> , and redeploy

End-to-End Flow — Both Labs Combined



Each step in both lab sections builds on the previous one, progressively expanding your understanding from simple function authoring to full event-driven automation orchestration.

Best Practices Demonstrated in These Labs

1

Least Privilege via Managed Roles

Using the pre-provisioned **LabRole** instead of root credentials or over-permissioned custom roles models real-world IAM best practices for least-privilege access control.

2

Infrastructure as Behavior

The EventBridge + Lambda pattern treats **infrastructure state as something to be enforced programmatically**, not managed manually – a foundational DevOps and FinOps principle.

3

Observability by Default

Every Lambda function automatically streams logs to **CloudWatch Logs**. Always verify execution via CloudWatch before assuming success – logs are your ground truth.

4

Deploy Before You Test

In both Node.js and Python Lambda functions, **clicking Deploy is a mandatory step** after editing code. Unsaved changes are not reflected in live executions – a common source of confusing test failures.

Quick Reference — Key Configuration Values

Lab	Resource Name	Key Setting
Section 4	BriefTextConverterService	Runtime: Node.js 18.x · Role: LabRole
Section 4	LegalServicePublicIngressAPI	Type: REST API · Method: POST · Proxy: ✓
Section 4	Deployment Stage	Stage name: prod
Section 5A	Sampleapp-env	Version label: v2-vocational-update-release
Section 5B	AutomatedInstanceStopController	Runtime: Python 3.9 · Role: LabRole
Section 5B	OneMinuteComputeHostCheckerCron	Rate: 1 minute · Target: Lambda · Role: LabRole

Labs Complete — Key Takeaways

Serverless = No Infrastructure

Lambda runs your code on demand. You pay only for actual execution time, and AWS manages all underlying compute — no servers to patch, scale, or maintain.

API Gateway = Public Front Door

API Gateway translates HTTP requests into Lambda invocations, handling authentication, routing, throttling, and CORS — giving your function a production-ready URL instantly.

EventBridge = Automated Scheduling

Combining EventBridge with Lambda creates powerful, cost-effective automation pipelines that run on a schedule — no always-on server required to execute recurring tasks.

Beanstalk = Managed PaaS

Elastic Beanstalk bridges the gap between raw EC2 and full serverless, giving teams a familiar deployment model with managed auto-scaling, health monitoring, and rolling updates.